



IA como copiloto para Infraestrutura

Material de Apoio do Curso

Alura — AI-IaC

2026

Sumário

Introdução ao Curso	4
Objetivo	4
Estrutura do Curso	4
Pré-requisitos	4
Como usar este material	5
Introdução à IA Generativa para Profissionais de Infra	6
Visão Geral	6
Visão geral da carreira AIOps	6
O que é AIOps	6
O papel do profissional	6
Pontos-chave	6
O que são LLMs e como funcionam	6
Conceitos fundamentais	6
Por que LLMs “alucinam”	7
Lab prático	7
O cenário atual de IA em operações de TI	7
Onde IA já é produção em operações	7
O que ainda é experimental	7
Dados de mercado	8
Casos de uso reais em DevOps e SRE	8
Categorias de uso	8
Lab prático	8
Configurando seu ambiente de trabalho	8
Subindo o ambiente com Vagrant	8
Configurando o acesso ao k3s no host	9
Diretórios sincronizados	10
Componentes do ambiente	10
Validação do ambiente	10
Conceito: ambiente de experimentação seguro	11
Prompt Engineering para Operações	12
Visão Geral	12
Fundamentos de prompt engineering	12
A anatomia de um prompt eficaz	12
Prompt ruim vs. prompt bom	12

Lab prático	13
Prompts para troubleshooting de infraestrutura	13
Template de troubleshooting	13
Conceito: chain-of-thought	13
Labs práticos	14
Prompts para análise de logs e métricas	14
Workflow de análise	14
Técnicas de pré-processamento	14
Labs práticos	14
Prompts para geração de manifests e IaC	15
Princípios para geração eficaz	15
Template para geração de Helm chart	15
Labs práticos	15
Biblioteca pessoal de prompts operacionais	16
Conceito: “prompt como código”	16
Estrutura recomendada	16
Labs práticos	16
Comparando como cada IA pensa	16
Modelos mentais diferentes	16
Como testar	17
Labs práticos	17

IA Aplicada a Terraform e Helm **18**

Visão Geral	18
Gerando código Terraform com assistentes de IA	18
Workflow de geração	18
Prompt de geração	18
Checklist de revisão do plan	18
Comandos essenciais	19
Labs práticos	19
Pontos-chave	19
Refatorando código Terraform com ajuda de IA	19
O problema de refatoração	19
Refactoring seguro	20
Workflow de refatoração	20
Labs práticos	20
IA para Helm charts e templates	21
Anatomia de um Helm chart	21
Complexidades de Go templates	21
Validação obrigatória	21
Labs práticos	21
Code review de Terraform assistido por IA	22
IA como “primeiro reviewer”	22
Prompt de review	22
Labs práticos	22
Pipeline completo de IaC com assistente integrado	23
O pipeline	23
Etapas do pipeline	23

Cenário prático	23
Labs práticos	23
Pontos-chave	24
Agentic Workflows e Limites da IA	25
Visão Geral	25
Chatbots vs. Agentes de IA	25
A diferença fundamental	25
Componentes de um agente	25
Por que isso muda o risco	25
Labs práticos	26
Para onde o mercado de agentes caminha	26
Os 3 principais frameworks	26
Comparativo resumido	27
Conceito: autonomia gradual	27
Labs práticos	27
Riscos de IA em produção: alucinações	27
Tipos de alucinação em laC	27
Pipeline de defesa contra alucinações	28
Checklist pós-geração	28
Labs práticos	28
Blast radius e guardrails	28
As 5 camadas de proteção	28
Princípio de menor privilégio para agentes	29
Trade-off: autonomia vs. segurança	29
Labs práticos	29
Construindo uma cultura de IA responsável no time	29
Framework de governança	29
Labs práticos	30
Resumo: o que levar da Aula 4	30
Referência Rápida	31
Comandos essenciais	31
Kubernetes (k3s)	31
Terraform	31
Helm	31
Assistentes de IA	32
Estrutura de prompt (template universal)	32
Checklist de validação (código gerado por IA)	32
Terraform	32
Helm	32
Kubernetes manifests	33
Mapa de labs	33

Introdução ao Curso

Objetivo

Nosso objetivo é capacitar profissionais de infraestrutura a usar IA generativa de forma prática, segura e integrada ao dia a dia de operações. Ao final do curso, vamos ser capazes de:

- Gerar IaC (Terraform e Helm) com assistentes de IA
- Fazer troubleshooting assistido por IA
- Revisar código com IA como “primeiro reviewer”
- Montar workflows agenticos com senso crítico e governança

Estrutura do Curso

O curso está dividido em quatro aulas:

Aula	Tema	Foco
1	Introdução à IA Generativa para Infra	Fundamentos, LLMs, ambiente
2	Prompt Engineering para Operações	Comunicação eficaz com LLMs
3	IA aplicada a Terraform e Helm	Geração, refatoração, pipeline
4	Agentic Workflows e limites da IA	Agentes, riscos, governança

Pré-requisitos

- Computador com Linux (ou WSL)
- Conta configurada no Kiro CLI
- Conta configurada no Gemini CLI
- Ambiente k3s (via Vagrant — instruções na Aula 1)

Como usar este material

Este PDF acompanha o curso e serve como referência rápida. Cada capítulo corresponde a uma aula e contém:

- Explicação dos conceitos principais
- Referência aos scripts e labs práticos
- Resumo dos pontos-chave para fixação

Os labs práticos estão no diretório `ai-iac-labs/` do repositório do curso.

Introdução à IA Generativa para Profissionais de Infra

Visão Geral

Nesta aula vamos explorar os fundamentos da IA generativa no contexto de operações de TI. Nosso foco será entender como LLMs funcionam (sem jargão de ML), conhecer o cenário atual do mercado e montar um ambiente de trabalho completo para os labs seguintes.

Visão geral da carreira AIOps

O que é AIOps

AIOps (Artificial Intelligence for IT Operations) é a aplicação de inteligência artificial para automatizar e melhorar operações de TI. Diferente de automação tradicional (scripts que seguem regras fixas), AIOps usa modelos que aprendem padrões e tomam decisões em cenários não previstos.

O papel do profissional

O papel do profissional de infra neste novo cenário **não é substituição, é alavancagem**. A IA amplifica a capacidade do operador — ele ainda precisa de julgamento, contexto organizacional e responsabilidade.

Pontos-chave

- Vagas de DevOps/SRE já pedem experiência com ferramentas de IA generativa
- AIOps não é novo — a novidade é a qualidade e acessibilidade dos modelos atuais
- O caminho é prático: vamos gerar IaC, fazer troubleshooting, revisar código, montar pipelines

O que são LLMs e como funcionam

Conceitos fundamentais

Conceito	Explicação
Token	Unidade mínima de texto (~4 caracteres). O modelo processa tokens, não palavras.
Janela de contexto	Quantidade máxima de tokens que o modelo “vê” por vez (ex: 128K tokens).
Temperatura	Controla a aleatoriedade. Baixa (0.1) = respostas determinísticas. Alta (1.0) = respostas criativas.
Predição de próximo token	O modelo gera texto prevendo qual token mais provavelmente vem depois do anterior.

Por que LLMs “alucinam”

Alucinações não são bugs — são consequência da arquitetura. O modelo gera texto **estatisticamente plausível**, não factualmente correto. Ele não “sabe” coisas; ele prevê sequências de texto com base em padrões aprendidos.

Implicação prática: nunca vamos confiar cegamente no output. Sempre vamos validar com ferramentas reais (`terraform validate`, `kubectl apply --dry-run`).

Lab prático

Referência: `ai-iac-labs/aula-1/04-demo-temperatura.md`

Demonstração do efeito de temperatura — mesmo prompt, respostas diferentes conforme o parâmetro.

Referência: `ai-iac-labs/aula-1/05-demo-contexto.md`

Demonstração de como a janela de contexto impacta a qualidade da resposta.

O cenário atual de IA em operações de TI

Onde IA já é produção em operações

- **Geração de código** (Copilot-style): assistentes gerando IaC, Dockerfiles, manifests
- **Análise de incidentes:** correlação de logs e métricas assistida por IA
- **Documentação:** geração de runbooks e READMEs a partir de código
- **Sumarização:** resumo de alertas, changelogs, post-mortems

O que ainda é experimental

- Agentes autônomos com permissão de execução em produção
- Rollback automático 100% decidido por IA
- Geração de arquitetura completa sem revisão humana

Dados de mercado

Pesquisas como o State of DevOps e relatórios Gartner/Forrester mostram adoção crescente, com foco em:

- Redução de tempo de resolução de incidentes (MTTR)
- Velocidade de provisionamento
- Padronização de código

Casos de uso reais em DevOps e SRE

Categorias de uso

Categoria	Exemplo	Quando usar
Geração	Helm chart, Terraform module, Dockerfile	Tarefas repetitivas e boilerplate
Análise	Diagnóstico de logs, correlação de métricas	Cenários com muitos dados
Sumarização	Runbook, changelog, post-mortem	Documentação e comunicação
Decisão assistida	Sugestão de causa raiz, opções de remediação	Incidentes e troubleshooting

Lab prático

Referência: [ai-iac-labs/aula-1/1.4/06-casos-uso-infra.md](#)

Exemplos concretos de IA aplicada a 3 cenários: geração de IaC, análise de incidente e documentação.

Configurando seu ambiente de trabalho

Subindo o ambiente com Vagrant

O ambiente de lab usa uma VM Ubuntu 24.04 com tudo pré-instalado (k3s, Helm, Terraform, Kiro CLI). O Vagrantfile está disponível em [material-de-apoio/ai-iac-labs/aula1/1.5/Vagrantfile](#).

Pré-requisitos na máquina host:

- [VirtualBox](#) instalado
- [Vagrant](#) instalado

Subindo a VM:

```
# Navegar até o diretório com o Vagrantfile
cd material-de-apoio/ai-iac-labs/aula1/1.5/

# Criar e provisionar a VM (primeira vez leva ~5-10 min)
vagrant up

# Acessar a VM
vagrant ssh
```

Configurando o acesso ao k3s no host

Após subir a VM, vamos acessar o cluster Kubernetes diretamente do host (sem precisar estar dentro da VM). O script k3s-setup.sh copia o kubeconfig da VM para nossa máquina local:

```
# Executar na raiz do projeto (onde está o Vagrantfile)
./k3s-setup.sh
```

O que o script faz:

1. Conecta na VM via `vagrant ssh`
2. Copia o arquivo `/etc/rancher/k3s/k3s.yaml` para `~/.kube/config` no host
3. Ajusta permissões do arquivo (`chmod 600`)
4. Valida a conexão com `kubectl cluster-info`

Depois de executar, vamos poder usar `kubectl` e `helm` direto do terminal do host:

```
kubectl get nodes          # Deve mostrar o node Ready
kubectl get pods -A        # Lista pods de todos os namespaces
helm list -A               # Lista releases Helm
```

Nota: Vamos executar `k3s-setup.sh` novamente se reiniciarmos a VM (`vagrant halt + vagrant up`), pois o IP pode mudar.

O que o provisionamento instala automaticamente:

- k3s (Kubernetes leve, single-node)
- Helm 3
- Terraform
- Node.js (via `fnm`)
- Kiro CLI (binário copiado de `./bin/`)
- Gemini CLI (via `npm`)

Comandos úteis do Vagrant:

```
vagrant up          # Criar/ligar a VM
vagrant ssh         # Acessar a VM via SSH
vagrant halt        # Desligar a VM (preserva dados)
vagrant destroy     # Destruir a VM completamente
vagrant provision   # Re-executar o provisionamento
vagrant status      # Ver status da VM
```

Portas mapeadas (host → VM):

Porta host	Porta VM	Serviço
8060	80	HTTP (Traefik)
8443	443	HTTPS
6443	6443	API Kubernetes
30000-30002	30000-30002	NodePorts para labs

Diretórios sincronizados

O Vagrant sincroniza automaticamente:

- ./ai-iac-labs → /home/vagrant/ai-iac-labs (labs do curso)
- ./bin → /home/vagrant/bin (binários como Kiro CLI)

Isso significa que podemos editar arquivos no host (com nosso editor preferido) e eles estarão disponíveis dentro da VM.

Componentes do ambiente

Ferramenta	Propósito	Verificação
k3s	Cluster Kubernetes local para labs	kubectl get nodes
Helm	Empacotamento de aplicações K8s	helm version
Terraform	Gerenciamento declarativo de infra	terraform version
Kiro CLI	Assistente de IA no terminal	which kiro-cli
Gemini CLI	Assistente alternativo para comparação	which gemini

Validação do ambiente

Referência: ai-iac-labs/aula-1/01-validacao-ambiente.sh

Script que verifica todas as dependências de uma vez:

```
cd ~/ai-iac-labs/aula-1  
bash 01-validacao-ambiente.sh
```

O script testa: kubectl, k3s, helm, terraform, Kiro CLI, Gemini CLI, CoreDNS, Traefik e conectividade.

Conceito: ambiente de experimentação seguro

O k3s local é um **sandbox** — erros não têm consequência. Isso nos permite experimentar livremente com IA sem risco. Este conceito de “blast radius zero” será expandido na Aula 4.

Prompt Engineering para Operações

Visão Geral

Nesta aula vamos aprender a comunicar intenção de forma precisa para LLMs. Vamos construir prompts estruturados para troubleshooting, análise de logs e geração de manifests, e organizar uma biblioteca reutilizável de prompts operacionais.

Fundamentos de prompt engineering

A anatomia de um prompt eficaz

Todo prompt para operações segue uma estrutura de 5 componentes:

Papel → Contexto → Instrução → Restrições → Formato de saída

Componente	O que faz	Exemplo
Papel	Define a persona do modelo	"Engenheiro SRE sênior"
Contexto	Informações do ambiente	"Cluster k3s, namespace prod"
Instrução	O que queremos que o modelo faça	"Diagnostiche o CrashLoop"
Restrições	Limites e regras	"Sem sugerir restart sem motivo"
Formato	Como entregar	"Lista ordenada por probabilidade"

Prompt ruim vs. prompt bom

Ruim:

cria um deployment kubernetes

Bom:

Papel: Engenheiro Kubernetes sênior.
Contexto: Cluster k3s single-node, namespace production.
Instrução: Crie um Deployment para nginx com 2 réplicas.
Restrições: Incluir liveness probe, resource limits (256Mi/200m).
Formato: YAML válido pronto para kubectl apply.

Lab prático

Referência: ai-iac-labs/aula-2/2.1/01-prompt-ruim.md e ai-iac-labs/aula-2/2.1/02-prompt-bom.md

Comparação direta entre prompts sem e com estrutura — observe a diferença de qualidade no output.

Referência: ai-iac-labs/aula-2/2.1/03-exercicio-prompt-passo-a-passo.md

Exercício guiado para construir seu primeiro prompt estruturado.

Prompts para troubleshooting de infraestrutura

Template de troubleshooting

Papel: SRE sênior com experiência em Kubernetes.

Contexto:

- Ambiente: k3s single-node, namespace [NAMESPACE]
- Sintoma: [DESCREVER O PROBLEMA]
- Desde quando: [TEMPO]

Dados disponíveis:

[COLAR LOGS OU OUTPUT RELEVANTE]

O que já tentei: [AÇÕES JÁ REALIZADAS]

Instrução: Identifique a causa raiz mais provável.

Formato: Lista de hipóteses ordenadas por probabilidade, com próximo passo para cada uma.

Conceito: chain-of-thought

Pedir que o modelo “raciocine passo a passo” antes de responder melhora significativamente a qualidade do diagnóstico. Adicionar “Antes de responder, explique seu raciocínio” ao prompt força o modelo a estruturar o pensamento.

Labs práticos

Referência: [ai-iac-labs/aula-2/exemplos-comparativos/02-troubleshooting-estruturado.md](#)

Exemplo de troubleshooting com e sem estrutura.

Referência: [ai-iac-labs/aula-2/logs-simulados/crashloop.log](#)

Log simulado de CrashLoopBackOff para praticar diagnóstico com IA.

Prompts para análise de logs e métricas

Workflow de análise

Nosso fluxo para análise de logs com IA:

1. **Filtrar antes de enviar** — não cole 500 linhas; filtre por severidade e janela temporal
2. **Pré-processar** — remova ruído (timestamps repetidos, linhas de debug irrelevantes)
3. **Prompt focado** — peça algo específico (correlação, padrões, anomalias)
4. **Validar** — confronte a análise da IA com dados reais

Técnicas de pré-processamento

```
# Filtrar últimos 5 minutos, só erros
kubectl logs <pod> --since=5m | grep -i error

# Remover linhas de debug
kubectl logs <pod> | grep -v DEBUG

# Extrair apenas timestamps e mensagens
kubectl logs <pod> | awk '{print $1, $NF}'
```

Labs práticos

Referência: [ai-iac-labs/aula-2/2.3/prompt-logs-performance.md](#)

Prompt estruturado para análise de performance em logs.

Referência: [ai-iac-labs/aula-2/2.3/prompt-logs-correlacao.md](#)

Prompt para correlação de eventos entre serviços.

Referência: [ai-iac-labs/aula-2/2.3/logs-simulados/](#)

Logs simulados para praticar diferentes cenários de análise.

Referência: [ai-iac-labs/aula-2/exemplos-comparativos/03-analise-logs-preprocessado.md](#)

Exemplo mostrando a diferença entre enviar log bruto vs. pré-processado.

Prompts para geração de manifests e IaC

Princípios para geração eficaz

1. **Seja específico em versões** — nunca aceitar :latest, pedir tag exata
2. **Nomeie recursos explicitamente** — evitar nomes genéricos gerados pelo modelo
3. **Defina padrões de segurança** — probes, limits, RBAC devem ser parte do prompt
4. **Peça estrutura de arquivos** — Chart.yaml, values.yaml, templates/ separados

Template para geração de Helm chart

***Papel:** Engenheiro de plataforma com experiência em Helm.*

Contexto: Cluster k3s, Helm 3.x.

Gere um Helm chart com:

- Nome: *[NOME]*
- Imagem: *[REGISTRY/IMAGEM:TAG]*
- Porta: *[PORTA]*
- Replicas: *[MIN-MAX]* (HPA baseado em CPU *[THRESHOLD]*%)
- Probes: liveness/readiness em *[ENDPOINT]*
- Secrets: via envFrom de Secret *[NOME-SECRET]*
- Limits: *[MEM]* RAM, *[CPU]* CPU

Formato: Estrutura completa de diretório.

Restrições: Sem valores hardcoded no template – tudo via values.yaml.

Labs práticos

Referência: [ai-iac-labs/aula-2/2.4/prompt-kubernetes-deployment.md](#)

Prompt para geração de Deployment Kubernetes completo.

Referência: [ai-iac-labs/aula-2/2.4/prompt-helm-chart.md](#)

Prompt para geração de Helm chart estruturado.

Referência: [ai-iac-labs/aula-2/2.4/prompt-dockerfile.md](#)

Prompt para geração de Dockerfile com boas práticas.

Referência: [ai-iac-labs/aula-2/2.4/prompt-runbook.md](#)

Prompt para geração de runbook operacional.

Biblioteca pessoal de prompts operacionais

Conceito: “prompt como código”

Prompts são artefatos versionáveis. Assim como IaC, prompts devem ser:

- Versionados (git)
- Categorizados (por caso de uso)
- Parametrizados (variáveis substituíveis)
- Documentados (descrição, exemplo de uso)
- Revisados (pelo time, como código)

Estrutura recomendada

```
prompts/  
├── troubleshooting/  
│   ├── crashloopbackoff.md  
│   └── oom-killed.md  
├── geracao/  
│   ├── helm-chart.md  
│   ├── dockerfile.md  
│   └── kubernetes-deployment.md  
└── analise/  
    ├── logs-correlacao.md  
    └── metricas-anomalia.md
```

Labs práticos

Referência: `ai-iac-labs/aula-2/2.5/01-demo-usar-template.md`

Demo: usar um template de prompt da biblioteca.

Referência: `ai-iac-labs/aula-2/2.5/02-demo-criar-prompt.md`

Demo: criar um novo prompt e adicioná-lo à biblioteca.

Referência: `ai-iac-labs/aula-2/prompts/`

Biblioteca completa de prompts operacionais do curso.

Comparando como cada IA pensa

Modelos mentais diferentes

Tipo	Comportamento	Melhor para
Arquiteto	Pensa antes de agir, pede clarificações, justifica	Problemas complexos, decisões arquiteturais
Executor	Responde rápido, assume o óbvio, vai direto ao código	Tarefas bem definidas, boilerplate

Como testar

Envie o mesmo prompt para Kiro CLI e Gemini CLI e observe:

- **Raciocínio:** raciocina passo a passo ou responde direto?
- **Construção:** gera tudo de uma vez ou em blocos?
- **Iteração:** reescreve tudo ou modifica cirurgicamente?

Labs práticos

Referência: `ai-iac-labs/aula-2/2.6/01-mesmo-prompt.md` até `ai-iac-labs/aula-2/2.6/07-diagnostico-com-etapas.md`

Sequência de exercícios comparando comportamento entre ferramentas de IA.

Referência: `ai-iac-labs/aula-2/EXERCICIO-COMPARATIVO.md`

Exercício guiado completo de comparação Kiro vs Gemini.

IA Aplicada a Terraform e Helm

Visão Geral

Nesta aula vamos colocar a mão na massa: vamos gerar e refatorar código Terraform e Helm charts com assistência de IA, integrar IA no code review e montar um pipeline completo de IaC no k3s.

Gerando código Terraform com assistentes de IA

Workflow de geração

Nosso fluxo de geração com IA vai seguir estas etapas:

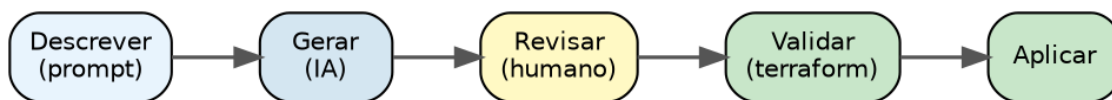


Figura 1: Fluxo de geração com IA

Regra fundamental: nunca aplicar terraform apply sem revisar o plan.

Prompt de geração

O prompt deve incluir:

- **Provider específico:** hashicorp/kubernetes
- **Caminho do kubeconfig:** /etc/rancher/k3s/k3s.yaml
- **Recursos desejados:** listar cada um com detalhes
- **Restrições:** variáveis para valores reutilizáveis, formato de arquivos

Checklist de revisão do plan

Antes de aplicar, verificar:

- ☐ Quantos resources serão criados? (coerente com o pedido?)
- ☐ Nomes estão corretos?
- ☐ Limits de CPU/memória fazem sentido?
- ☐ Namespace é o esperado?

- ☐ Não há destroy inesperado?

Comandos essenciais

```
terraform init      # Baixar providers
terraform validate  # Verificar sintaxe
terraform plan      # Ver o que será feito (sem fazer)
terraform apply     # Aplicar mudanças
terraform destroy   # Remover tudo
```

Labs práticos

Referência: [ai-iac-labs/aula-3/3.1/](#)

Lab completo de geração de Terraform com IA. Inclui:

- `prompt-geracao.md` — Prompt para gerar namespace + deployment + service + configmap + quota
- `prompt-rbac.md` — Prompt para iterar adicionando RBAC
- `prompt-converter-yaml.md` — Prompt para converter YAML existente em Terraform
- `main.tf`, `variables.tf`, `outputs.tf` — Código de referência
- `rbac.tf` — RBAC gerado na iteração

Referência: [ai-iac-labs/aula-3/01-namespace/main.tf](#)

Exemplo simples: criação de namespace com labels via Terraform.

Referência: [ai-iac-labs/aula-3/02-nginx-deploy/main.tf](#)

Exemplo intermediário: Deployment + Service.

Referência: [ai-iac-labs/aula-3/03-config-secret/main.tf](#)

Exemplo com ConfigMap + Secret + referências entre recursos.

Pontos-chave

- **IA como primeiro rascunho** — o output precisa de revisão antes do apply
- **Iteração** — começar simples e pedir incrementos funciona melhor que pedir tudo de uma vez
- **Terraform vs kubectl/Helm** — Terraform para recursos de plataforma (namespace, quota, RBAC); Helm para aplicações com releases frequentes

Refatorando código Terraform com ajuda de IA

O problema de refatoração

Código Terraform “legado” (escrito às pressas) apresenta:

- Valores hardcoded (sem variáveis)

- Secrets em plain text no código
- Imagens com :latest
- Falta de probes e resource limits
- Labels inconsistentes

Refactoring seguro

A distinção crítica em Terraform:

Tipo de mudança	Efeito	Risco
Mudar valor de variável	Update in-place	Baixo
Adicionar atributo	Update in-place	Baixo
Renomear resource	Destroy + Recreate	Alto
Mudar tipo de resource	Destroy + Recreate	Alto

Solução para renomear sem destruir: usar moved blocks:

```
moved {  
  from = kubernetes_deployment.app  
  to   = kubernetes_deployment.order_api  
}
```

Workflow de refatoração

1. Pedir análise à IA (code smells, segurança, boas práticas)
2. Classificar findings por severidade
3. Aplicar uma melhoria por vez
4. Validar com terraform plan que mostra 0 destroys
5. Verificar no k3s que o serviço continua rodando

Labs práticos

Referência: [ai-iac-labs/aula-3/3.2/](https://github.com/ai-iac-labs/aula-3/3.2/)

Lab de refatoração com código intencionalmente ruim. Inclui:

- main.tf — Código com problemas (secrets hardcoded, :latest, sem limits)
- prompt-analise.md — Prompt para análise de code smells
- prompt-refactor-incremental.md — Prompt para refatoração segura e incremental
- moved.tf — Exemplo de moved block para renomear sem destruir

IA para Helm charts e templates

Anatomia de um Helm chart

```
mychart/
├── Chart.yaml           # Metadados do chart
├── values.yaml          # Valores padrão (o "contrato" do chart)
├── templates/
│   ├── _helpers.tpl     # Funções auxiliares (labels, nomes)
│   ├── deployment.yaml  # Template do Deployment
│   ├── service.yaml     # Template do Service
│   └── hpa.yaml          # Template do HPA (opcional)
└── README.md            # Documentação
```

Complexidades de Go templates

Os pontos de falha mais comuns com IA gerando Helm templates:

- **Indentação** — `{{ toYaml .Values.resources | nindent 12 }}` (o número importa!)
- **Contexto (.)** — muda dentro de `{{ range }}` e `{{ with }}`
- **Conditionals** — `{{ if .Values.hpa.enabled }}` para features opcionais

Validação obrigatória

```
# Renderizar sem instalar (verificar output)
helm template test ./mychart

# Lint (verificar estrutura e boas práticas)
helm lint ./mychart

# Instalar com dry-run
helm install test ./mychart --dry-run
```

Labs práticos

Referência: [ai-iac-labs/aula-3/3.3/](#)

Lab de geração de Helm chart com IA, incluindo prompts e chart gerado.

Referência: [ai-iac-labs/aula-3/helm/04-helm-basic/](#)

Exemplo de chart básico.

Referência: [ai-iac-labs/aula-3/helm/05-helm-conditionals/](#)

Exemplo com conditionals (features opcionais via values).

Referência: [ai-iac-labs/aula-3/helm/06-helm-helpers/](#)

Exemplo com helpers customizados em `_helpers.tpl`.

Code review de Terraform assistido por IA

IA como “primeiro reviewer”

A IA é excelente para nos ajudar a capturar:

- Permissões excessivas em RBAC (verbs = ["*"])
- NetworkPolicies abertas
- Imagens sem tag fixa
- Secrets sem marcação sensitive
- Falta de probes e resource limits

A IA **não substitui** o review humano — ela não vê o state, não sabe o que roda em produção, e não entende políticas internas do time.

Prompt de review

Papel: *Security engineer sênior especializado em Kubernetes.*

Contexto: Código Terraform (provider hashicorp/kubernetes).
Será usado em produção.

Foco do review:

- Segurança: RBAC, NetworkPolicy, secrets
- Boas práticas: variáveis, referências, naming
- Resiliência: probes, limits, replicas
- Riscos: o que pode causar problemas em produção

Formato: Tabela | Severidade | Problema | Sugestão |

Labs práticos

Referência: [ai-iac-labs/aula-3/3.4/](#)

Lab de code review com código contendo problemas intencionais:

- `codigo-para-review.tf` — Código com problemas (Role com ["*"], NetworkPolicy aberta, etc.)
- `prompt-review.md` — Prompt estruturado para review
- `checklist-review.md` — Checklist manual para comparar com findings da IA
- `gabarito.md` — Gabarito com todos os problemas esperados
- `codigo-corrigido.tf` — Código após correções



Figura 2: Pipeline completo de IaC

Pipeline completo de IaC com assistente integrado

O pipeline

Neste pipeline vamos usar um **shell script como orquestrador**. Cada etapa tem um gate humano (read -p). A IA nos assiste em 3 papéis diferentes: geração, review e documentação.

Etapas do pipeline

#	Etapas	Ferramenta	Gate
1	Especificar	Linguagem natural	Humano define requisitos
2	Gerar	Kiro CLI	Humano revisa output
3	Validar	terraform init/validate/plan	Automático
4	Revisar	Kiro CLI (prompt de segurança)	Humano avalia findings
5	Corrigir	Editor + terraform plan	Humano corrige
6	Aplicar	terraform apply + kubectl	Automático + verificação
7	Documentar	Kiro CLI	Humano valida README
8	Reproduzir	destroy + apply	Prova que funciona do zero

Cenário prático

O pipeline cria toda a infra para um serviço payment-api:

- Namespace com ResourceQuota e LimitRange
- Deployment com probes, limits, secrets via envFrom
- Service + ConfigMap
- RBAC (ServiceAccount + Role + RoleBinding)
- NetworkPolicy restritiva

Labs práticos

Referência: ai-iac-labs/aula-3/3.5/pipeline.sh

Script principal do pipeline. Executa todas as 8 etapas com gates humanos. Uso: ./pipeline.sh

Referência: `ai-iac-labs/aula-3/3.5/valida.sh`

Script de validação — verifica se todos os resources existem no cluster após apply.

Referência: `ai-iac-labs/aula-3/3.5/prompts/`

Prompts usados em cada etapa do pipeline:

- `01-gerar.md` — Prompt de geração do código Terraform
- `02-review.md` — Prompt de code review de segurança
- `03-documentar.md` — Prompt de geração de documentação

Pontos-chave

1. **Pipeline ≠ CI tool** — são etapas com gates; o terminal é o runner
2. **IA em 3 papéis** — geração, review e documentação
3. **terraform plan como gate** — nenhum apply sem plan limpo
4. **Human-in-the-loop** — IA propõe, humano valida com ferramentas reais
5. **Reprodutibilidade** — destroy + apply prova que o código é autossuficiente
6. **De script para CI** — depois basta trocar `read -p` por aprovação no PR

Agentic Workflows e Limites da IA

Visão Geral

Nesta aula vamos entender a diferença entre chatbots e agentes, conhecer o ecossistema de frameworks, identificar riscos reais de IA em infraestrutura e implementar guardrails para uso responsável em produção.

Natureza da aula: teórica com exemplos práticos. O foco está em construir modelo mental, não em executar scripts.

Chatbots vs. Agentes de IA

A diferença fundamental

Aspecto	Chatbot	Agente
Output	Texto (sugestões)	Ações (executa comandos)
Loop	Pergunta → Resposta	Observar → Decidir → Agir → Observar
Risco	Texto errado é inofensivo	Ação errada pode derrubar produção
Exemplo	“Como faço deploy?”	“Faça deploy deste chart no k3s”

Componentes de um agente

Por que isso muda o risco

Um chatbot que gera YAML errado não causa dano — o humano ainda precisa copiar e aplicar. Um agente com tool use pode executar terraform apply diretamente. O **blast radius** é fundamentalmente diferente.

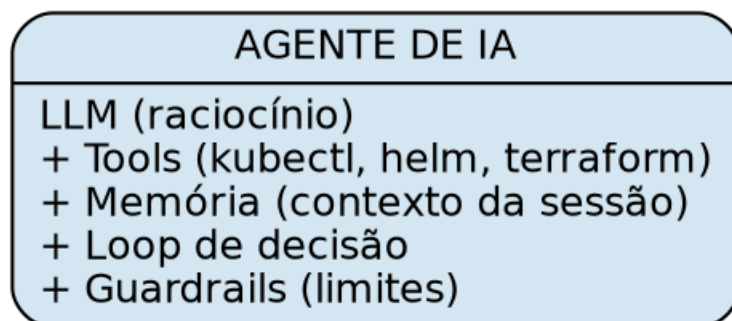


Figura 3: Componentes de um agente de IA

Labs práticos

Referência: ai-iac-labs/aula-4/4.1/README.md

Explicação detalhada com exemplos de chatbot vs. agente no contexto de operações.

Para onde o mercado de agentes caminha

Os 3 principais frameworks

LangChain

- **Foco:** Chains (sequências) + Agents (loops com tools)
- **Ponto forte:** Enorme ecossistema de integrações, comunidade ativa
- **Ponto fraco:** Complexidade alta, mudanças frequentes de API
- **Caso de uso em infra:** Agente que recebe alerta → consulta logs → correlaciona com deploys → sugere causa raiz

CrewAI

- **Foco:** Multi-agentes colaborativos com papéis diferentes
- **Ponto forte:** Simples de configurar, abstração de alto nível
- **Ponto fraco:** Menos controle fino, menos integrações
- **Caso de uso em infra:** Crew com agente “SRE” + agente “DevOps” + agente “Reviewer”

AutoGen (Microsoft)

- **Foco:** Conversação entre agentes (dialogam para resolver)
- **Ponto forte:** Fácil incluir humano no loop, bom para decisão colaborativa
- **Ponto fraco:** Overhead de conversa (lento, custoso em tokens)
- **Caso de uso em infra:** Grupo de agentes discutindo estratégia de migração

Comparativo resumido

Aspecto	LangChain	CrewAI	AutoGen
Foco	Chains + tools	Multi-agente	Conversação
Complexidade	Alta	Média	Média
Maturidade	Alta	Média	Média
Melhor para	Integrações complexas	Workflows com papéis	Decisão colaborativa

Conceito: autonomia gradual

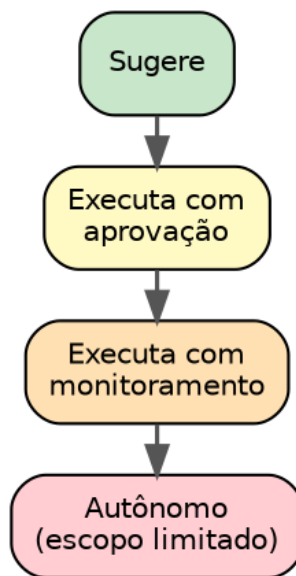


Figura 4: Autonomia gradual

Critério prático: blast radius baixo (teste) = mais autonomia. Blast radius alto (produção) = mais aprovação humana.

Labs práticos

Referência: ai-iac-labs/aula-4/4.2/README.md

Material aprofundado sobre frameworks, padrões de orquestração e quando usar cada um.

Riscos de IA em produção: alucinações

Tipos de alucinação em IaC

Tipo	Exemplo	Impacto
Provider inexistente	provider "aws_kubernetes"	Erro de init
API deprecated	apiVersion: extensions/v1beta1	Funciona hoje, quebra amanhã
Permissão excessiva	0.0.0.0/0 na porta 22	Vulnerabilidade de segurança
Valores sem sentido	replicas: 100 (quando queria 1)	Custo inesperado
Recurso inventado	Atributo que não existe no provider	Erro de validate

Pipeline de defesa contra alucinações

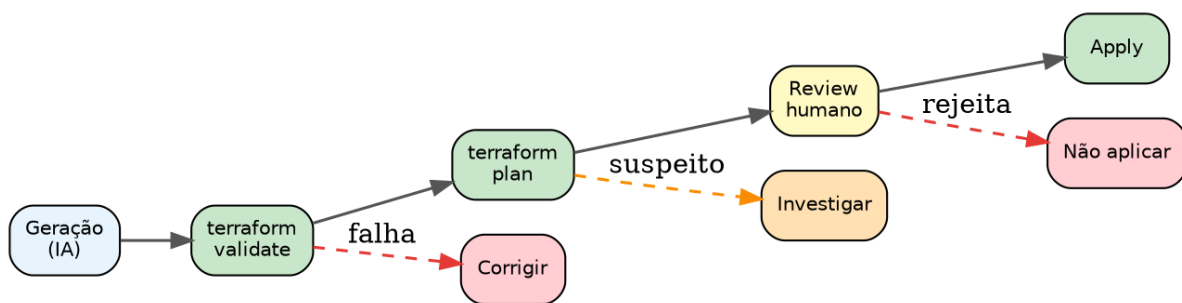


Figura 5: Pipeline de defesa contra alucinações

Checklist pós-geração

- ☐ terraform validate passa sem erros?
- ☐ Versões de API estão corretas e atuais?
- ☐ Permissões seguem princípio de menor privilégio?
- ☐ Valores numéricos fazem sentido (replicas, limits, quotas)?
- ☐ Não há secrets em plain text?
- ☐ Referências entre resources estão corretas?
- ☐ helm lint / helm template renderiza corretamente?

Labs práticos

Referência: [ai-iac-labs/aula-4/4.3/README.md](https://github.com/ai-iac-labs/aula-4/4.3/README.md)

Exemplos detalhados de alucinações, como detectá-las e checklist de defesa.

Blast radius e guardrails

As 5 camadas de proteção

Camada	O que faz	Exemplo
1. Sandbox	Ambiente isolado	k3s local, nunca produção
2. Dry-run	Simular sem executar	terraform plan, --dry-run=server
3. Aprovação	Humano decide antes de agir	read -p no pipeline, PR approval
4. Monitoramento	Observar após execução	Health checks, alertas
5. Rollback	Reverter se algo der errado	helm rollback, terraform apply do state anterior

Princípio de menor privilégio para agentes

Agentes de IA devem ter as **mesmas restrições** que um operador júnior:

- Sem acesso a produção diretamente
- Permissões limitadas ao namespace/escopo do trabalho
- Logs de todas as ações executadas
- Tempo limite (timeout) para execuções

Trade-off: autonomia vs. segurança

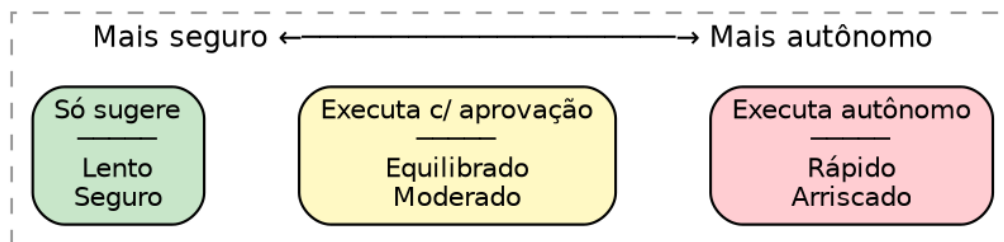


Figura 6: Trade-off autonomia vs. segurança

Recomendação: começar com “só sugere” e evoluir gradualmente conforme evidência.

Labs práticos

Referência: [ai-iac-labs/aula-4/4.4/README.md](https://ai-iac-labs.github.io/aula-4/4.4/README.md)

Detalhamento das 5 camadas com exemplos práticos de implementação.

Construindo uma cultura de IA responsável no time

Framework de governança

Zonas de uso

Zona	Permissão	Exemplo
Verde	Usar livremente	Geração de rascunhos, documentação, análise de logs
Amarela	Usar com review obrigatório	IaC para staging, Helm charts, RBAC
Vermelha	Requer aprovação + validação extra	Produção, networking, secrets, permissões

Responsabilidade

Regra clara: quem faz merge é responsável, independente de quem (ou o quê) gerou o código.

A IA não tem responsabilidade. O humano que aprova, aplica ou faz merge assume a responsabilidade integral pelo resultado.

Métricas de acompanhamento

Métrica	O que mede	Meta
Tempo economizado	Horas poupadas com assistência de IA	Rastrear tendência
Taxa de aceitação	% de output da IA usado sem alteração	Não tem “ideal”
Bugs por origem	Bugs de código humano vs. código assistido	Paridade ou melhor
Incidentes	Incidentes causados por output de IA	Zero

Labs práticos

Referência: ai-iac-labs/aula-4/4.5/README.md

Material completo sobre governança: política de uso, checklist de validação, template de CONTRIBUTING.md e métricas.

Resumo: o que levar da Aula 4

1. **Agentes ≠ chatbots** — execução real tem risco real
2. **Frameworks existem** mas maturidade varia — avaliar caso a caso
3. **Alucinações são inevitáveis** — o pipeline de defesa é obrigatório
4. **Guardrails em camadas** — sandbox → dry-run → aprovação → monitoramento → rollback
5. **Governança é gestão de risco** — não burocracia. Começa restrito, expande com evidência.

Referência Rápida

Comandos essenciais

Kubernetes (k3s)

kubectl get nodes	# Status do cluster
kubectl get all -n <namespace>	# Tudo em um namespace
kubectl describe pod <pod> -n <ns>	# Detalhes de um pod
kubectl logs <pod> -n <ns> --since=5m	# Logs recentes
kubectl apply --dry-run=server -f <file>	# Validar sem aplicar
kubectl create ns <namespace>	# Criar namespace

Terraform

terraform init	# Inicializar (baixar providers)
terraform validate	# Validar sintaxe
terraform plan	# Ver mudanças propostas
terraform plan -out=tfplan	# Salvar plan para apply exato
terraform apply tfplan	# Aplicar plan salvo
terraform apply -auto-approve	# Aplicar sem confirmação
terraform destroy	# Destruir todos os resources
terraform state list	# Listar resources no state

Helm

helm template <name> ./chart	# Renderizar templates (sem instalar)
helm lint ./chart	# Verificar estrutura e boas práticas
helm install <name> ./chart	# Instalar chart
helm install <name> ./chart --dry-run	# Simular instalação
helm upgrade <name> ./chart	# Atualizar release
helm rollback <name> <rev>	# Reverter para versão anterior
helm uninstall <name>	# Desinstalar
helm list	# Listar releases

Assistentes de IA

```
kiro chat "seu prompt aqui"      # Prompt direto no Kiro
kiro chat < prompt.md            # Enviar arquivo como prompt
gemini "seu prompt aqui"         # Prompt no Gemini CLI
```

Estrutura de prompt (template universal)

Papel: *[Persona que o modelo deve assumir]*

Contexto:

- Ambiente: *[cluster, namespace, ferramentas]*
- Situação: *[o que está acontecendo]*

Instrução: *[O que queremos que o modelo faça]*

Restrições:

- *[Limitação 1]*
- *[Limitação 2]*

Formato de saída: *[Como entregar a resposta]*

Checklist de validação (código gerado por IA)

Terraform

- ☐ terraform validate passa?
- ☐ terraform plan mostra apenas mudanças esperadas?
- ☐ Sem secrets em plain text?
- ☐ Imagens com tag fixa (não :latest)?
- ☐ Resource limits definidos?
- ☐ Variáveis para valores reutilizáveis?
- ☐ Referências entre resources (não strings hardcoded)?

Helm

- ☐ helm lint passa sem warnings?
- ☐ helm template renderiza corretamente?
- ☐ values.yaml documenta todas as opções?
- ☐ Indentação correta nos templates?
- ☐ Probes configurados?
- ☐ Resources limits definidos?

Kubernetes manifests

- ☐ `kubectl apply --dry-run=server` aceita?
- ☐ Labels consistentes?
- ☐ Namespace correto?
- ☐ Security context definido?
- ☐ Probes e limits presentes?

Mapa de labs

Aula	Lab	Tema	Arquivo principal
1	—	Validação do ambiente	aula-1/01-validacao-ambiente.sh
2	2.1	Prompt ruim vs bom	aula-2/2.1/
2	2.3	Análise de logs	aula-2/2.3/
2	2.4	Geração de manifests	aula-2/2.4/
2	2.5	Biblioteca de prompts	aula-2/2.5/
2	2.6	Comparativo Kiro vs Gemini	aula-2/2.6/
3	3.1	Geração Terraform	aula-3/3.1/
3	3.2	Refatoração Terraform	aula-3/3.2/
3	3.3	Helm charts com IA	aula-3/3.3/
3	3.4	Code review com IA	aula-3/3.4/
3	3.5	Pipeline completo	aula-3/3.5/pipeline.sh
4	4.1-4.5	Agentes e governança	aula-4/4.*/README.md